



南開大學
Nankai University

计算机学院
软件工程实验报告

实验 3: 智能城市生成系统——软件设计报告

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 5 月 9 日

目录

1 用例图	2
1.1 用例图说明	3
1.2 核心参与者 (Actors) 的职责与定位	3
1.3 子系统包及用例说明	4
1.4 包含关系说明	4
2 活动图	4
2.1 (一) 主活动图 (多模态交互核心流程)	5
2.1.1 主活动图说明	5
2.2 (二) 智能语言解析与仿真子活动图	6
2.2.1 自然语言子活动图说明	6
2.3 (三) 草图驱动与布局重构子活动图	7
2.3.1 草图驱动子活动图说明	7
3 类图	8
3.1 类图说明	8
4 顺序图	9
4.1 (一) 业务层: 智能语言解析与仿真控制时序	9
4.1.1 业务层顺序图说明	10
4.2 (二) 系统层: 基于图像草图的布局重构时序	10
4.2.1 系统层顺序图说明	11
4.3 (三) 数据层: 模板化资产联动加载时序	11
4.3.1 数据层顺序图说明	12
5 协作图	12
5.1 (一) 智能语言交互与仿真协作图	13
5.1.1 协作图说明	13
5.2 (二) 基于图像草图的布局重构协作图	14
5.2.1 协作图说明	14
5.3 (三) 核心资产模板加载协作图	15
5.3.1 协作图说明	15
6 状态图	16
6.1 状态图说明	16
7 构件图	17
7.1 构件图说明	18
8 部署图	19
8.1 部署图说明	19
9 总结	20

智能城市生成系统——软件设计分析报告

1 用例图

用例图说明智能城市生成系统对外提供哪些能力：有哪些参与者、各自触发哪些用例，以及用例之间如何用 `<<include>>` 组合起来。整体上覆盖三类用户（场景建模师、行业分析师、系统管理员），以及四类常见输入方式：自然语言、二维草图、坐标点集和风格模板；系统在完成解析与配置后，还要完成资产加载、布局生成、仿真与渲染等环节并形成可反复修改的闭环。

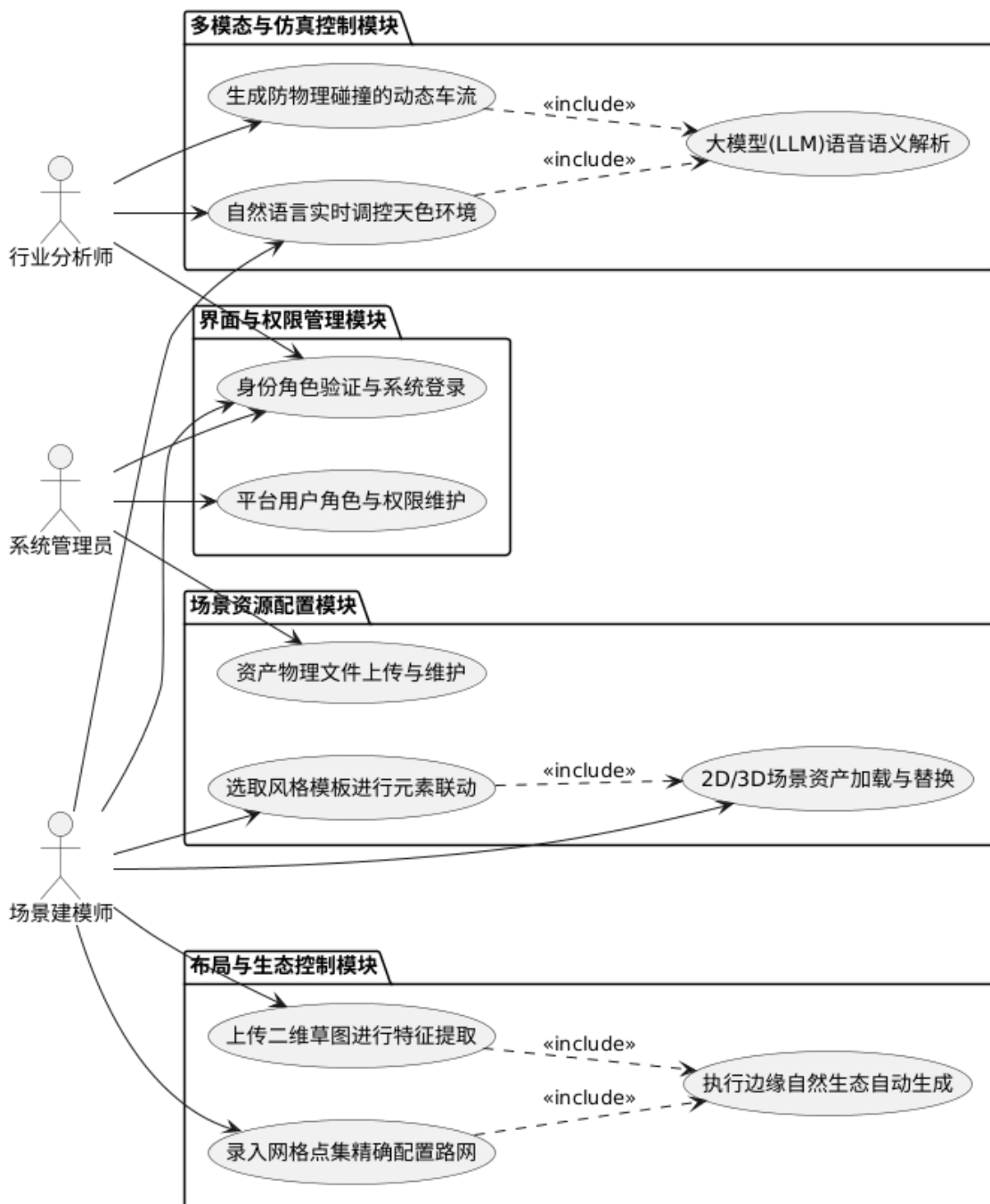


图 1.1: 用例图

1.1 用例图说明

• 与系统目标的对应

- 系统希望提高搭建效率、支持布局与外围生态、支持自然语言交互、支持车流与人群等动态仿真，并具备登录与后台管理能力。
- 图中四个包 packet 大致对应这些方向：场景资源配置侧重资产与模板；布局与生态控制侧重路网、草图与生态生成；多模态与仿真控制侧重语言解析、环境与仿真；界面与权限管理侧重登录与权限维护。

• 角色分工

- 场景建模师熟悉三维制作，但希望少做重复摆放类工作，同时仍要能精细改布局。
- 行业分析师更关心推演与演示，希望用简单方式改环境、跑动态效果。
- 系统管理员负责账号、资产与平台侧维护。
- 用例图里，建模师连接布局、资产以及部分环境与语言类用例；分析师主要连接环境与车流仿真；管理员主要连接资产上传与权限维护，以减少无关功能干扰、降低误操作风险。

• 输入方式在图中的落点

- 自然语言相关能力集中在「大模型 (LLM) 语音语义解析」，并被「自然语言实时调控天色环境」「生成防物理碰撞的动态车流」所包含。
- 草图与点集走布局包，并在成功后包含生态生成。
- 模板选择走「选取风格模板进行元素联动」，并包含具体的资产替换。
- 多种输入最后都落实到具体用例上，便于实现和测试。

1.2 核心参与者 (Actors) 的职责与定位

• 场景建模师

- 登录后可使用：资产加载与替换、风格模板联动、点集配置路网、草图上传与特征提取，以及用语言调节天色等。
- 既支持自动化批量工作，又保留点集、草图等对布局更可控的入口。

• 行业分析师

- 侧重用场景做方案演示与分析，一般不操作底层网格编辑。
- 用例图里主要保留「自然语言实时调控天色环境」和「生成防物理碰撞的动态车流」，两者都包含「大模型 (LLM) 语音语义解析」，表示自然语言要先完成语义理解，再驱动环境与仿真。
- 不必了解 Blender 脚本细节。

• 系统管理员

- 负责资产文件入库与分类维护，以及用户与角色的后台配置；与其他角色一样需要先登录。
- 监控接口状态、日志审计等如需要，可在实现阶段作为补充能力。

1.3 子系统包及用例说明

- 界面与权限管理模块

- 所有参与者先完成「身份角色验证与系统登录」，系统再按角色展示功能。
- 管理员使用「平台用户角色与权限维护」维护账号与权限。
- 在调用语言解析、资产替换等操作之前先确认身份与权限。

- 场景资源配置模块

- 「资产物理文件上传与维护」由管理员维护模型与贴图等文件。
- 「2D/3D 场景资产加载与替换」用于在场景中替换具体实例。
- 「选取风格模板进行元素联动」用于按主题批量联动替换，并通过 `<<include>>` 触发资产替换，避免只改界面而不改场景内容。

- 布局与生态控制模块

- 「录入网格点集精确配置路网」用于精确控制路网。
- 「上传二维草图进行特征提取」用于从平面图提取结构信息。
- 两者在成功后都 `<<include>>` 「执行边缘自然生态自动生成」，表示主城区结构变化后，外围山体、水体、绿化等应随边界一起更新。

- 多模态与仿真控制模块

- 环境与车流两类用例都 `<<include>>` 「大模型 (LLM) 语音语义解析」。
- 自然语言驱动的操作应统一走语义解析，再交给渲染与仿真执行。

1.4 包含关系说明

- **模板与资产替换**：选择模板要落到具体模型与贴图的替换；包含关系强调模板操作与资产替换绑定在一起。
- **环境与仿真与语言解析**：用语言改环境或启停仿真时，应先完成解析再执行，避免跳过语义理解直接改场景。
- **布局与生态**：点集或草图生成布局后包含生态生成，强调生态是布局结果的配套步骤。

2 活动图

活动图描述一件工作如何一步步推进，以及界面、主程序、Blender 各自负责哪一段。下面三张图分别对应：整体多模态主流程、自然语言解析与仿真分支、草图驱动布局分支。

2.1 (一) 主活动图（多模态交互核心流程）

从登录开始，经多种输入、统一处理，再到 Blender 更新场景并回传画面；用户不满意可以返回修改，满意后再保存。

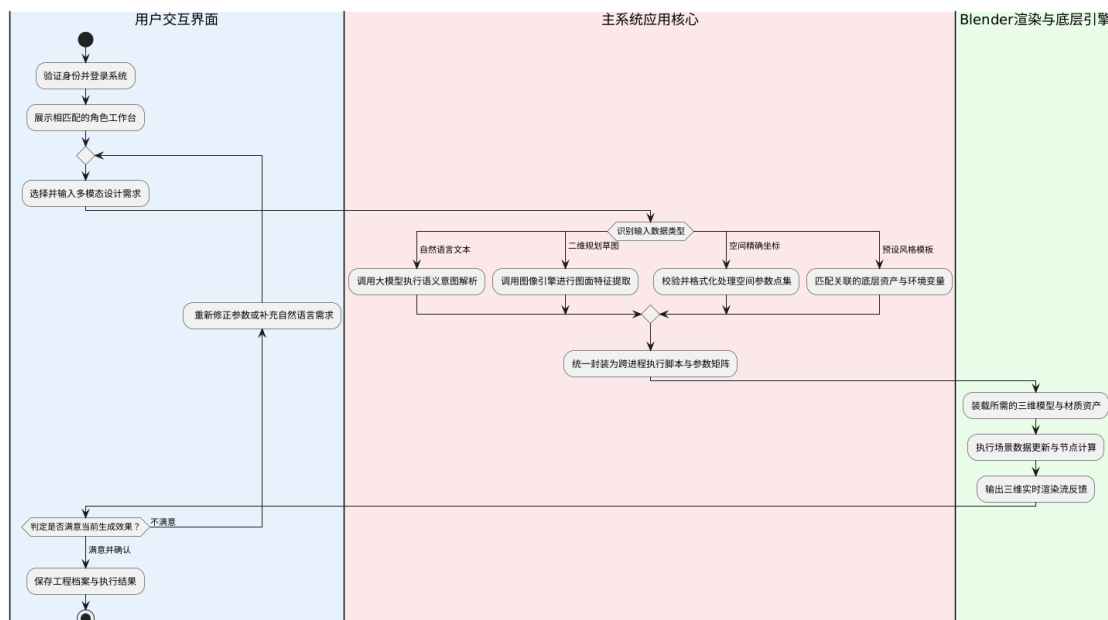


图 2.2: 主活动图（多模态交互核心流程）

2.1.1 主活动图说明

• 登录与工作台

- 用户先验证身份并登录，系统按角色打开对应工作台。

• 按输入类型分支

- 主程序根据输入类型分别处理：自然语言走大模型语义解析；二维草图走图像特征提取；坐标点集做格式校验与整理；风格模板则匹配资产与环境变量。
- 四种方式最后都要能落到同一套可执行的脚本与参数上，便于交给 Blender。

• 统一封装

- 各分支的结果封装成跨进程可用的脚本与参数，把不同来源的意图转成 Blender 能执行的指令。

• Blender 执行与回传

- 加载所需模型与材质，更新场景并计算，再把实时渲染结果返回界面。
- 系统依赖 Blender 完成三维更新与显示。

• 不满意则重来

- 用户对效果不满意时，可回到前面修改参数或补充描述；满意则保存工程并结束，体现常见的建模迭代方式。

- 对于实现上可以注意的点

- 登录后再做语言、图像、仿真等耗时操作，便于记录操作者。
- 用分支合并减少重复流程。
- 对 Blender 的输入尽量采用稳定的数据格式，便于测试与复现。
- 返回修改的路径可减少错误参数一路传到渲染端。

2.2 (二) 智能语言解析与仿真子活动图

专门展开“自然语言 → 大模型 → 生成脚本 → Blender 执行”的过程，并区分改环境和跑仿真两类结果。

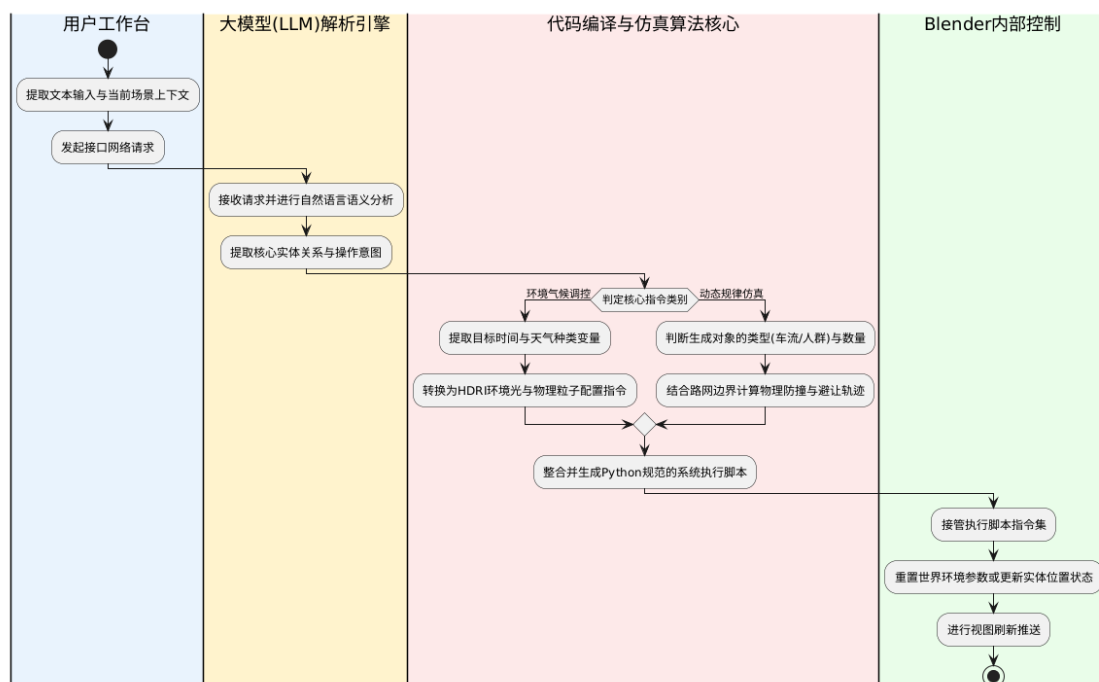


图 2.3: 智能语言解析与仿真子活动图

2.2.1 自然语言子活动图说明

- 带上场景信息再请求

- 工作台把用户输入连同当前场景相关信息一起发给大模型服务，减少理解偏差，例如当前天气、已加载范围、道路概况等。

- 大模型输出意图

- 大模型返回实体与操作意图等结构化信息，后续再生成可在 Blender 侧执行的脚本。

- 分支：环境还是仿真

- 若指令偏环境，则整理时间、天气等变量，对应到环境光、粒子等设置。
- 若指令偏仿真，则要确定车流或人群等对象类型与数量，并结合路网等信息计算路径与避让。

- 大模型负责理解与参数归纳，具体轨迹与规则仍由仿真与几何逻辑完成。

• Blender 执行

- Blender 接收脚本后更新世界环境或实体状态，并刷新视图。

• 实现补充

- 大模型调用受网络与耗时影响，界面侧适合有等待提示与超时处理。
- 自然语言应限制在城市场景相关说法内；无法解析时应明确提示用户，而不是静默失败。

2.3 (三) 草图驱动与布局重构子活动图

描述上传城市规划草图后的识别、校验、生成建筑与道路，以及外围生态更新。

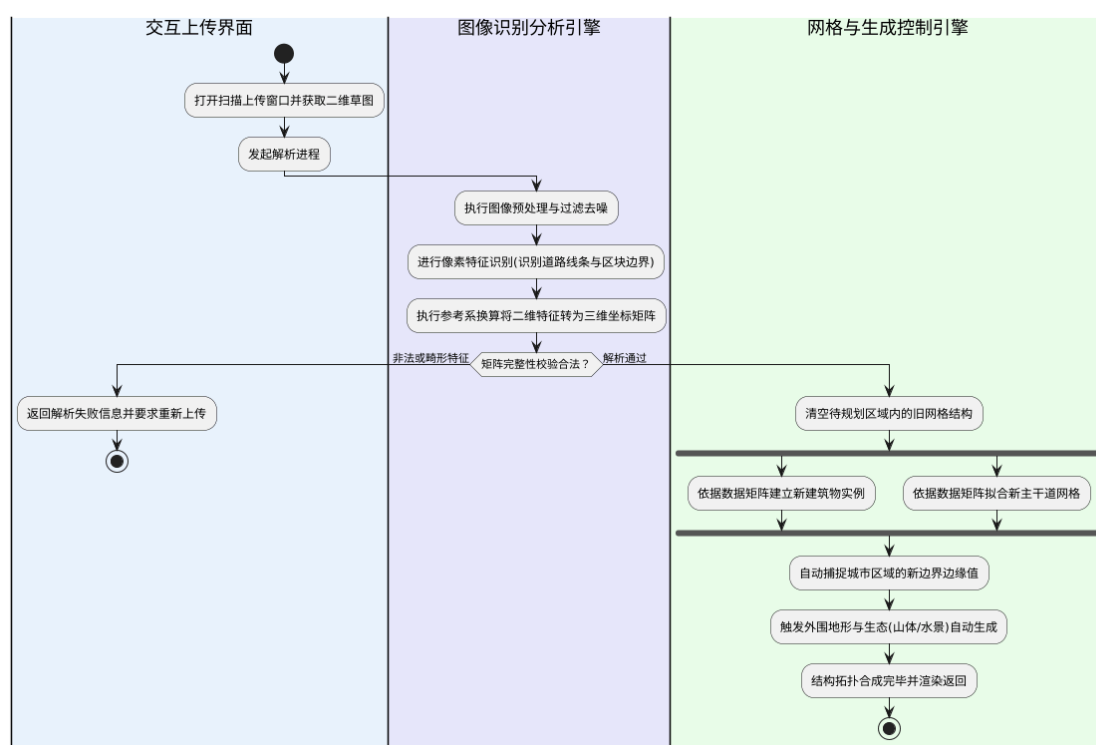


图 2.4: 草图驱动与布局重构子活动图

2.3.1 草图驱动子活动图说明

• 上传与识别

- 图像先做预处理与去噪，再识别道路线与区块边界，把二维信息换算到三维坐标矩阵。
- 草图不能当作最终几何，必须经过识别与换算。

• 参考系要固定

- 换算时要统一比例、方向和原点，否则生成结果容易整体偏移或比例不对。

• 校验失败则停止

– 若矩阵不合法，回到上传界面提示重新上传，避免把错误数据交给 Blender。

• 并行生成建筑与道路

- 校验通过后清空待建区域旧结构，再并行生成建筑实例与道路网格，缩短等待时间。
- 实现时要注意数据一致性与资源竞争。

• 更新边界与生态

- 根据新的城区边界生成外围地形与生态，与用例图中“布局包含生态”的含义一致。

3 类图

类图描述系统里主要的数据与职责：有哪些类、关键字段与方法，以及类之间如何关联。它把用户、工程方案、资产、指令、仿真实体和渲染接口几类概念分开，方便分工实现。

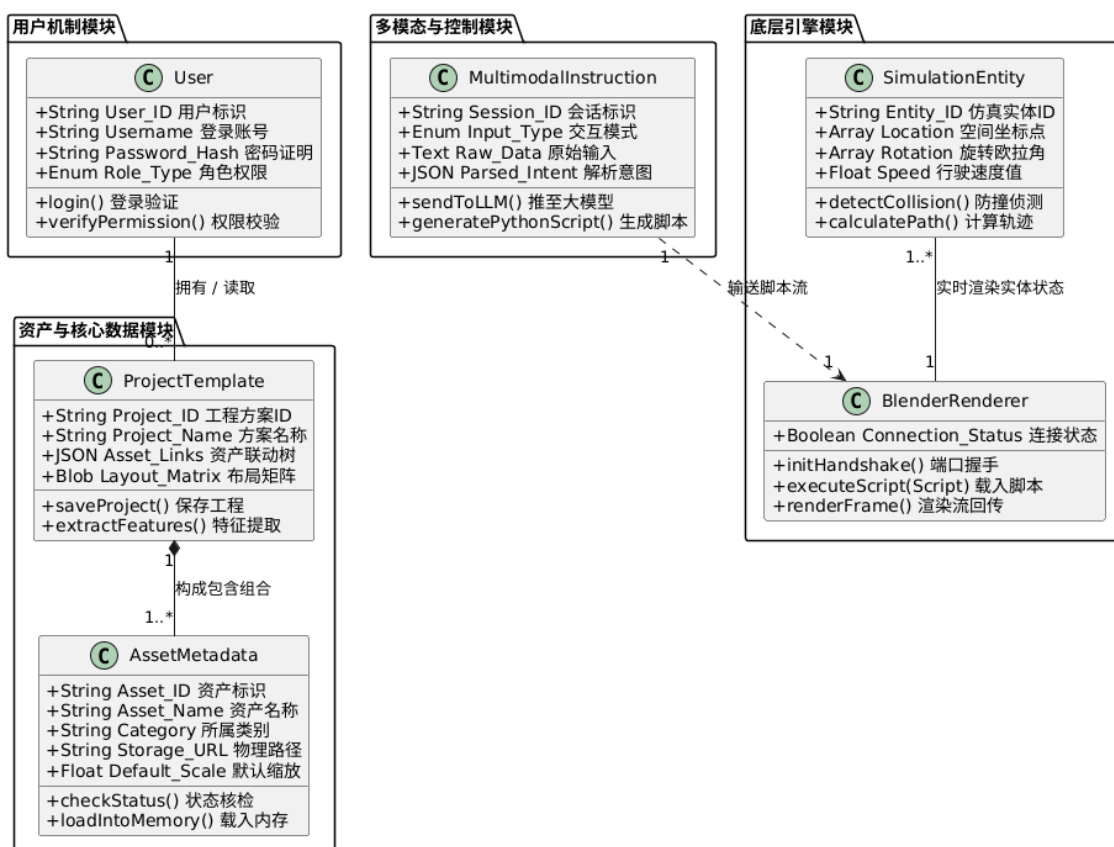


图 3.5: 类图

3.1 类图说明

• 用户机制模块 User

- 表示登录用户与角色。
- `Role_Type` 与 `verifyPermission()` 用于按角色判断能否执行某操作。
- `Password_Hash` 表示口令应以哈希形式存储。

- `User` 与 `ProjectTemplate` 为一对多：一个用户可以有多个工程方案。
- 资产与核心数据 `AssetMetadata`、`ProjectTemplate`
 - `AssetMetadata` 描述单个资源的路径、类别与默认缩放等，并提供状态检查与加载。
 - `ProjectTemplate` 保存方案名称、资产联动信息与布局矩阵等，并提供保存与特征提取相关操作。
 - 工程方案与多个资产之间是组合关系：方案由若干资产条目构成，删除或调整方案时要注意资产引用关系。
- 多模态与控制 `MultimodalInstruction`
 - 保存输入类型、原始内容与解析后的意图。
 - 提供调用大模型与生成 Python 脚本的方法。
 - 对 `BlenderRenderer` 是依赖关系：指令生成后交给渲染侧执行，而不是由指令类长期持有渲染器。
- 底层引擎 `SimulationEntity`、`BlenderRenderer`
 - `SimulationEntity` 表示仿真中的对象及其位置、速度等，并提供碰撞检测与路径计算。
 - `BlenderRenderer` 负责与 Blender 建立连接、执行脚本与回传帧，用连接状态表示是否已就绪。
- 设计取舍
 - 类图不展开 Blender 内部全部对象类型，把脚本执行集中在 `executeScript` 一类接口上，具体由插件实现。
 - 解析结果与联动信息若用 JSON 等半结构化字段保存，需要约定格式版本，并对大模型返回内容做校验，减少非法字段导致执行失败。

4 顺序图

顺序图按时间顺序画出对象之间的消息：谁先调用谁、返回什么。下面三条分别对应自然语言改场景、草图重构布局、模板批量换资产。

4.1 （一）业务层：智能语言解析与仿真控制时序

对应自然语言调节环境或启停车流仿真的典型调用过程。

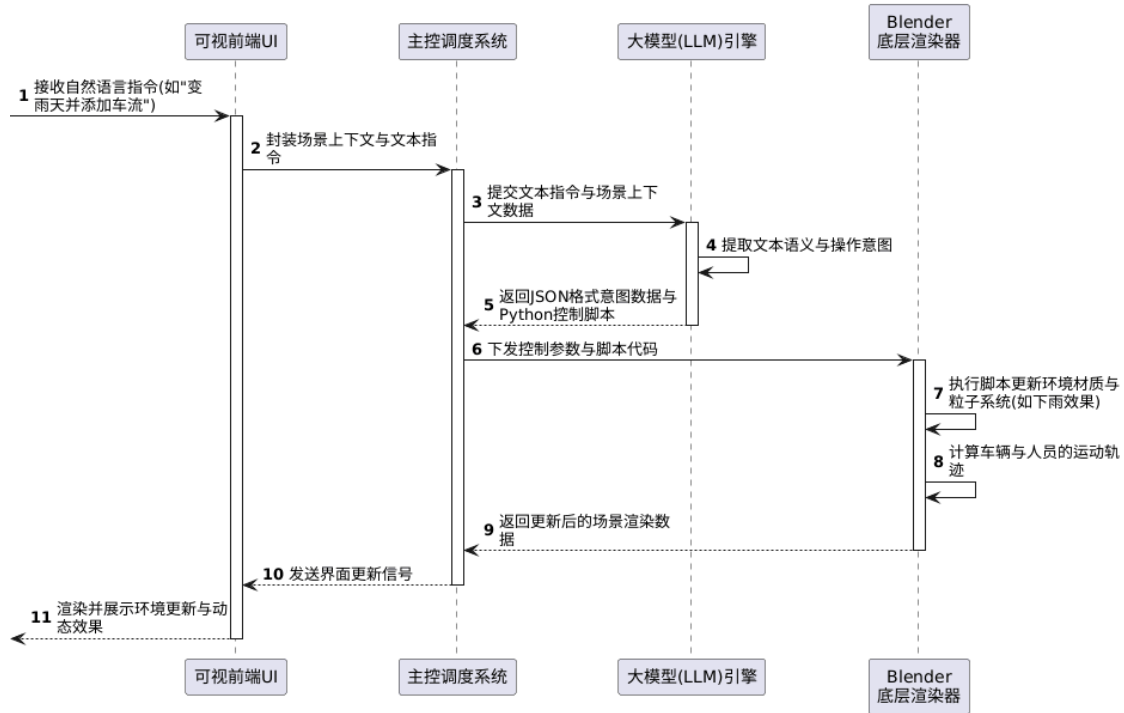


图 4.6: 顺序图：智能语言解析与仿真控制时序

4.1.1 业务层顺序图说明

- 界面接收用户输入，可做基本校验并收集当前场景快照。
- 主控模块把文本与场景上下文打包发给大模型服务。
- 大模型返回结构化意图以及可在 Blender 中执行的 Python 脚本（或等价指令）；结构化结果便于界面说明与记录，脚本用于实际改场景。
- Blender 执行脚本：例如更新材质与粒子表现雨天，或计算车辆与行人的运动。
- 更新结果回到主控与界面并显示。
- 若大模型超时或返回内容无法解析，主控应向界面返回明确错误，而不是继续执行。

4.2 （二）系统层：基于图像草图的布局重构时序

对应上传草图后识别、查资产、再在 Blender 中重建几何的过程。

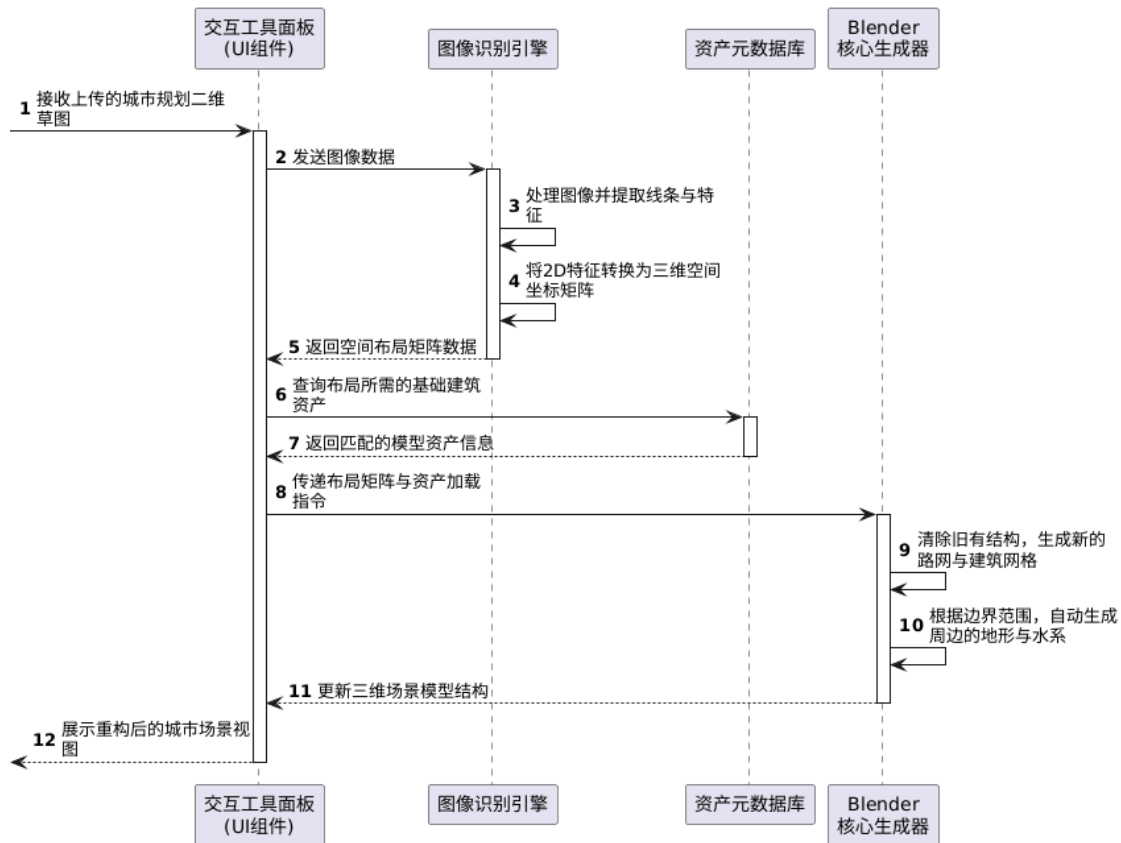


图 4.7: 顺序图：基于图像草图的布局重构时序

4.2.1 系统层顺序图说明

- 图像模块处理草图并输出三维布局矩阵，而不是只把原图传给 Blender。
- 根据布局需要查询资产库，拿到要实例化的建筑等资源信息。
- Blender 清除旧结构，生成新路网与建筑，并按边界补充地形与水体等。
- 图中先查库再驱动 Blender，表示先确定用哪些资产，再实例化；若改为在 Blender 内懒加载，需要重新梳理依赖关系。

4.3 （三）数据层：模板化资产联动加载时序

对应选择风格模板后批量核对与替换资产的过程。

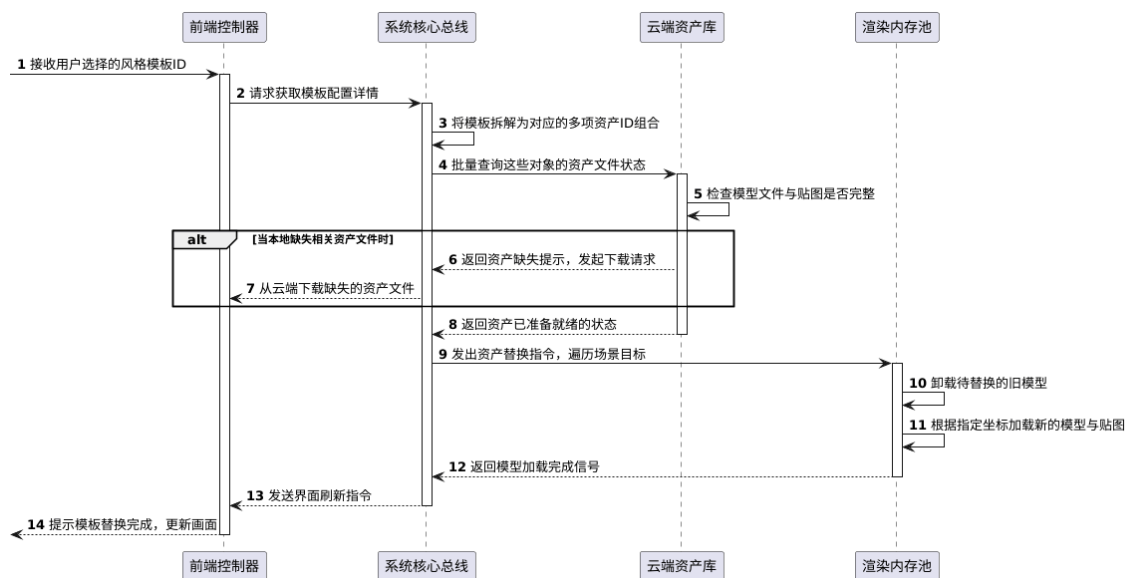


图 4.8: 顺序图：模板化资产联动加载时序

4.3.1 数据层顺序图说明

- 前端提交模板标识，核心服务把模板拆成多条资产编号。
- 向存储侧批量核对文件是否齐全；若有缺失则提示并下载补齐。
- 在渲染侧内存中卸载旧实例，再按位置装载新模型与贴图。
- 完成后通知前端刷新界面。
- 缺失资产时界面应清楚提示，避免场景只换了一半。

5 协作图

协作图用带序号的消息说明对象之间的协作关系。与顺序图相比，它更突出**对象之间如何连成一张协作网**，以及**每条有编号的消息在业务上代表什么**。下面三张图分别对应自然语言驱动、草图布局、模板加载三条链路。

5.1 (一) 智能语言交互与仿真协作图

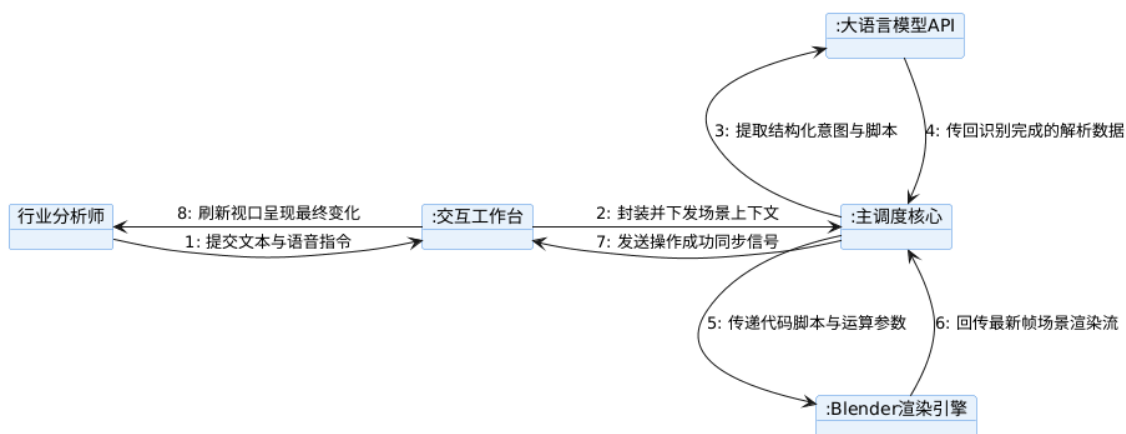


图 5.9: 协作图：智能语言交互与仿真

5.1.1 协作图说明

• 协作链路 with 编号含义

- 第 1 步：行业分析师向「交互工作台」提交文本或语音类指令，这是人机交互的唯一入口，便于做权限校验与操作留痕。
- 第 2 步：工作台把指令与当前场景相关信息打包交给「主调度核心」，由主调度统一决定后续是否调用大模型、何时调用 Blender。
- 第 3、4 步：主调度与大语言模型接口交互，完成语义理解并取回结构化结果或可执行脚本；这两步把“理解自然语言”与“驱动三维软件”解耦，大模型可替换供应商而不改 Blender 侧主流程。
- 第 5、6 步：主调度把脚本与参数交给「Blender 渲染引擎」，由其实际改环境或更新仿真实体，并把最新画面或状态回传。
- 第 7、8 步：主调度通知工作台刷新，工作台再向分析师呈现结果，形成闭环。

• 与系统角色的关系

- 分析师不直接连接 Blender，降低学习成本，也避免绕过业务规则随意改场景。
- 主调度处于中心位置，适合集中记录日志、限流、重试以及与安全策略相关的判断。

• 与顺序图的关系

- 消息内容与第四章第一条顺序图一致，只是强调对象两两之间的链接关系。
- 审查设计时可用本图检查：是否出现工作台绕开主调度直连 Blender 的路径；若存在，需要评估鉴权与一致性是否仍能保障。

• 异常与扩展

- 大模型不可用或超时，主调度应在第 3、4 步附近失败并回到工作台提示用户，而不是静默卡住。
- 若以后增加“仅内部 API、不走大模型”的快捷通道，也建议仍经主调度转发，以保持协作结构清晰。

5.2 (二) 基于图像草图的布局重构协作图

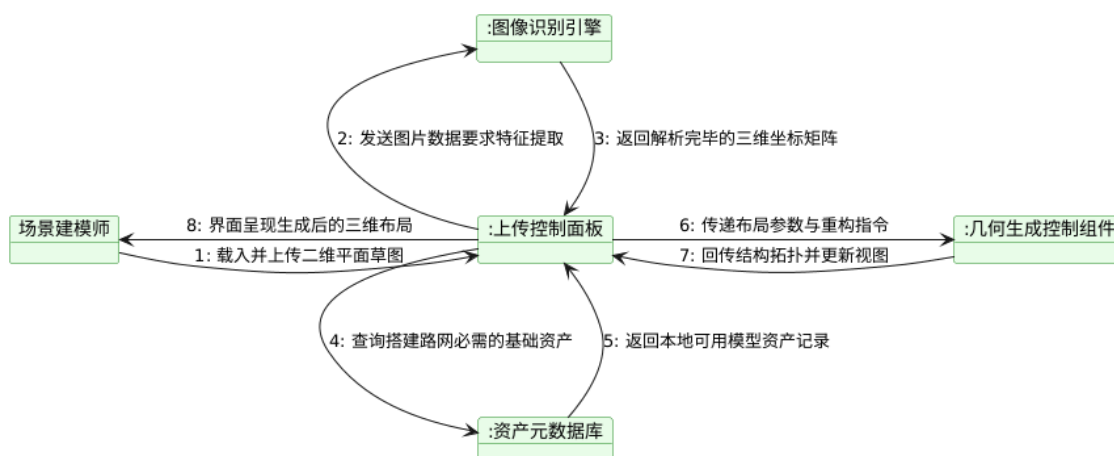


图 5.10: 协作图：基于图像草图的布局重构

5.2.1 协作图说明

• 协作链路及编号含义

- 第 1 步：场景建模师在「上传控制面板」载入或拍摄平面草图，触发一次布局重构流程。
- 第 2、3 步：面板把图像数据交给「图像识别引擎」，引擎返回三维布局矩阵（或等价结构数据）；这一步把“像素级草图”变成“可计算的几何约束”，是草图驱动的关键。
- 第 4、5 步：面板根据矩阵向「资产元数据库」查询所需建筑、道路等资源的标识与路径，数据库返回本地或云端可用的记录；没有这一步，后续生成往往不知道实例化哪一类资产。
- 第 6、7 步：面板把布局参数与资产选择结果交给「几何生成控制组件」，由其实际改场景拓扑并把更新结果回传面板；在实现上该组件通常对应 Blender 插件或渲染侧服务。
- 第 8 步：建模师在界面上看到新的三维布局。

• 为何把识别与查库分开

- 识别侧重算法与模型版本迭代，资产库侧重元数据与文件管理；分开后更换识别方案时不必大改资产表结构，反之亦然。
- 面板处于中间，负责把“识别输出”和“资产记录”拼成一份完整生成指令，避免识别引擎直接依赖存储细节。

• 与顺序图的关系

- 与第四章第二条顺序图中的 UI、Vision、DB、Blender 四类参与者对应；协作图里「几何生成控制组件」与顺序图中的 Blender 生成职责相同，名称不同不影响理解。

• 数据质量与反馈

- 若第 3 步返回的矩阵不可靠，面板应提示重新上传或手工修正参数，而不是强行进入第 6 步。
- 若第 5 步发现资产缺失，应在面板明确提示缺哪些资源，与模板加载流程中的缺件处理思路一致。

5.3 （三）核心资产模板加载协作图

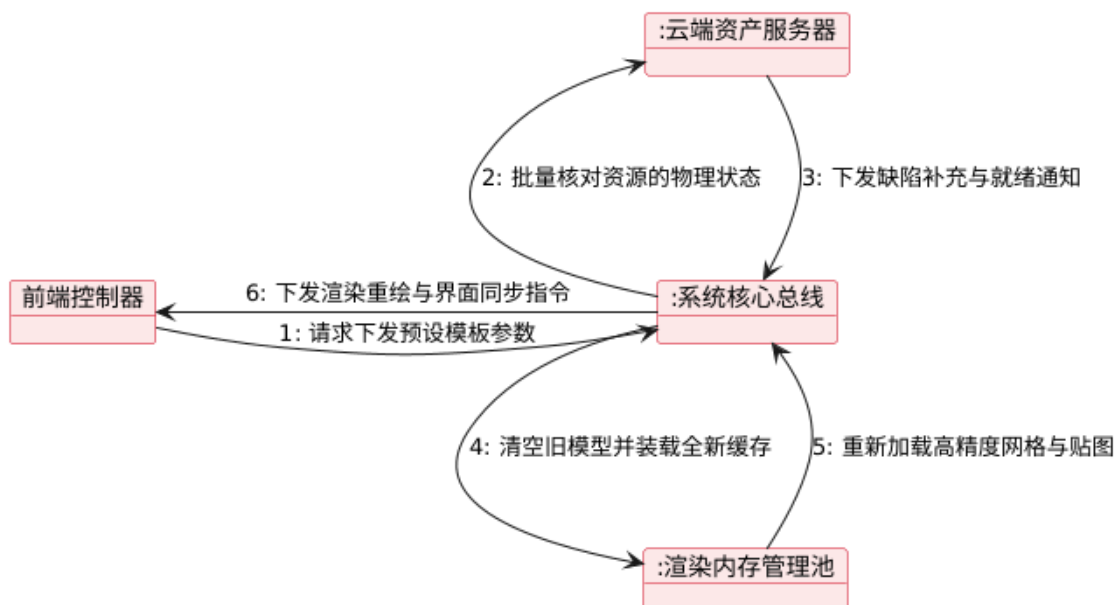


图 5.11: 协作图：核心资产模板加载

5.3.1 协作图说明

• 协作链路 with 编号含义

- 第 1 步：「前端控制器」向「系统核心总线」请求应用某个预设模板，通常只带模板编号或名称。
- 第 2、3 步：总线向「云端资产服务器」批量核对模板涉及的文件是否齐全、版本是否可用；服务器返回缺件列表或全部就绪的通知。
- 第 4、5 步：总线向「渲染内存管理池」发出替换指令：先卸载旧实例释放显存与场景引用，再按模板配置装载新网格与贴图；完成后向总线回报。
- 第 6 步：总线通知前端刷新界面与状态，让用户看到模板已生效。

• 图中没有画最终用户的原因

- 本图侧重后台数据与资源如何流动；用户操作已体现在第 1 步的前端请求里。
- 与用例「选取风格模板进行元素联动」对应：用户一点模板，背后是多构件协作完成校验、下载与内存替换。

• 与部署与存储的对应

- 云端资产服务器在部署上常对应对象存储或资产网关；内存池对应渲染节点或 Blender 进程内的资源缓存。
- 总线角色强调：模板事务应由一处协调，避免前端直接对存储和渲染各发一套指令导致状态不一致。

• 一致性与体验

- 第 3 步若发现缺件，应阻止第 4 步的“半替换”，或在第 6 步前明确提示“部分资源未就绪”，与第四章第三条顺序图中的分支思想一致。
- 大批量替换时要注意内存峰值，可在总线侧做排队或分块替换，避免一次性载入过多高精度模型导致崩溃。

6 状态图

状态图描述一个工程从新建或载入到可编辑、仿真、保存与退出的主要状态，以及状态之间如何切换。

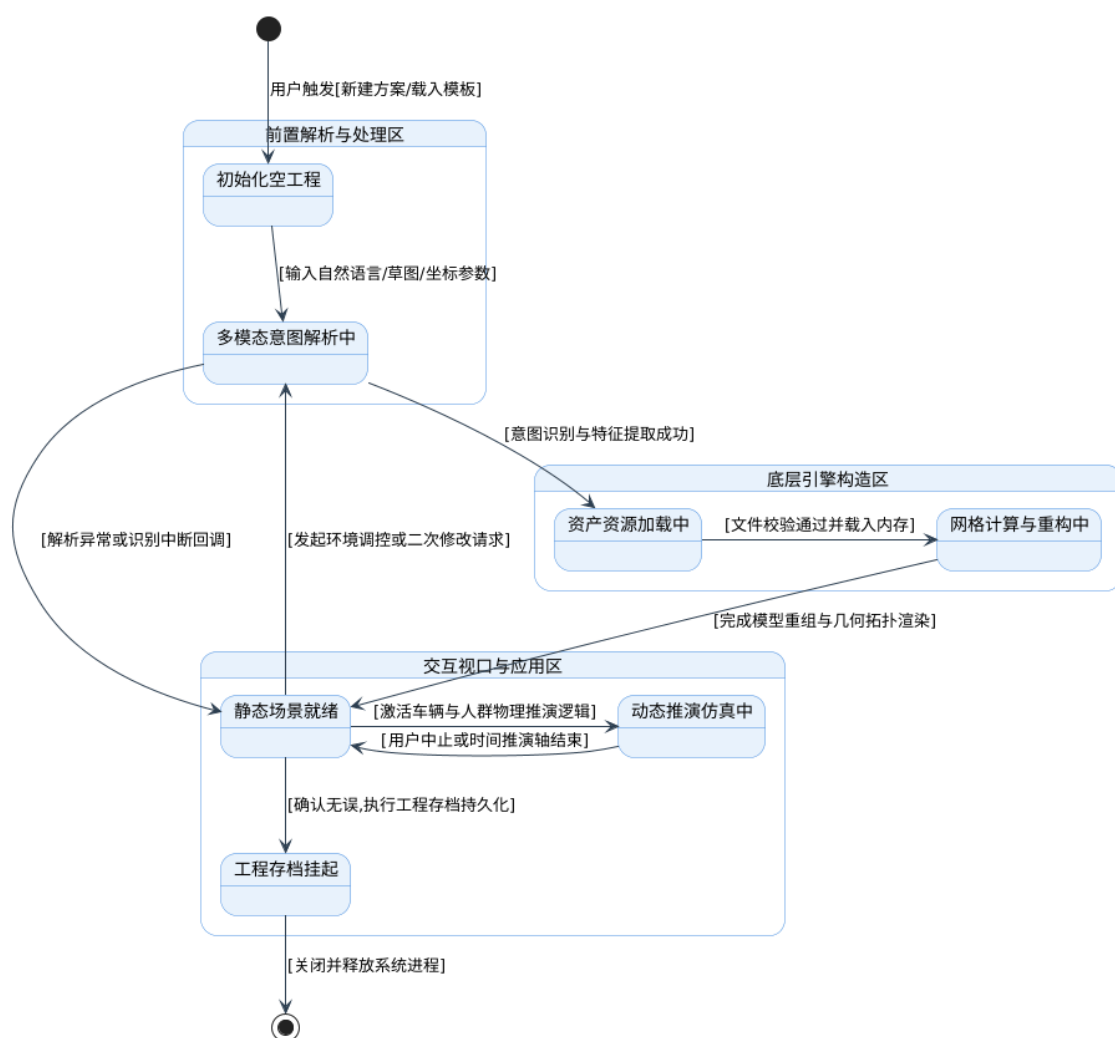


图 6.12: 状态图

6.1 状态图说明

• 整体结构

- 状态大致分为三块：前置解析与处理、底层引擎构造、交互视口与应用。
- 用户从新建或载入模板进入流程，中间可能多次在“解析—加载—就绪”之间往返，最后存档并退出。

- **前置解析与处理**

- 初始化空工程：建立会话与默认场景。
- 多模态意图解析中：汇总自然语言、草图或参数类输入的解析过程。
- 若解析异常，也可转入静态场景就绪，保留上一轮可用场景；是否采用这种降级方式可按产品要求定。

- **底层引擎构造**

- 资产资源加载中：从存储载入模型与贴图，并做完整性检查。
- 网格计算与重构中：根据布局或模板结果更新路网、建筑等拓扑。
- 完成后进入静态场景就绪。

- **交互视口与应用**

- 静态场景就绪：用户可浏览、发起再次修改或进入仿真。
- 动态推演仿真中：运行车流、人群等动态效果。
- 仿真结束可回到静态场景就绪。
- 工程存档挂起：用户确认后持久化工程。
- 就绪状态也可回到解析中，对应“改参数、再生成”的迭代。

- **结束与界面提示**

- 从工程存档挂起可关闭进程，释放资源。
- 按上述状态划分界面，便于显示“解析中”“加载中”“仿真中”等提示，并在相应阶段禁用或启用按钮、提供重试入口。

7 构件图

构件图从实现角度画出主要软件部件以及它们通过哪些接口相连。

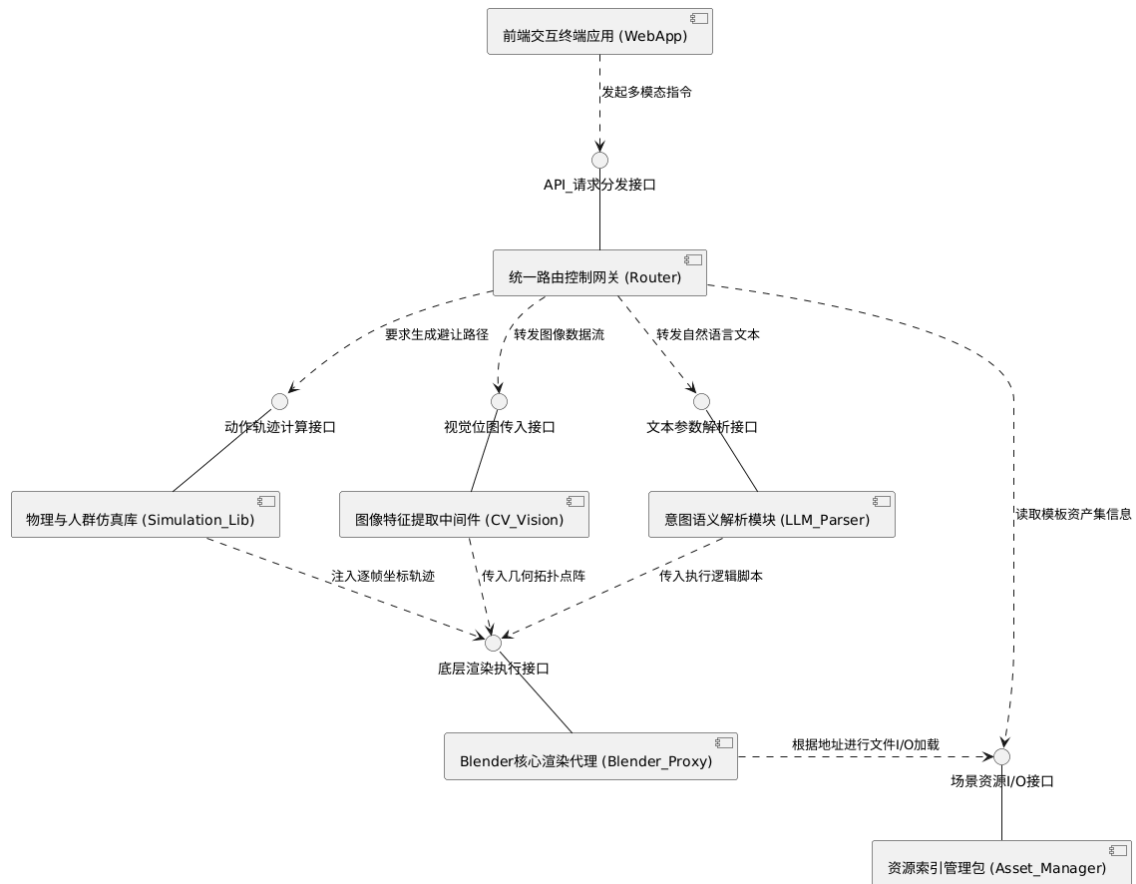


图 7.13: 构件图

7.1 构件图说明

• 前端与网关

- Web 前端通过「API 请求分发接口」访问「统一路由控制网关」。
- 网关作为统一入口，便于鉴权、限流和路由到不同后端能力。

• 四类业务能力构件

- 「意图语义解析模块」：处理自然语言，对外提供文本参数解析接口。
- 「图像特征提取中间件」：处理草图与图像，对外提供视觉位图传入接口。
- 「物理与人群仿真库」：计算轨迹与避让，对外提供动作轨迹计算接口。
- 「资源索引管理包」：管理资产元数据与读写调度，对外提供场景资源 I/O 接口。
- 网关按请求类型调用上述接口，而不是在网关内写死具体实现，便于替换模块或单独升级。

• Blender 核心渲染代理

- 提供「底层渲染执行接口」，供解析、视觉、仿真三类构件调用。
- 三类业务最终都要落到同一套渲染执行通道上，减少重复维护多套与 Blender 的连接逻辑。
- 代理在执行时仍通过资源接口查询路径与文件状态，保证载入内容与资产库一致。

• 结构带来的效果

- 资源管理与渲染代理分离：资源负责“有什么、在哪里”，代理负责“怎么执行进 Blender”。
- 语言解析独立成构件：更换大模型服务时主要改解析模块与网关配置。
- 仿真库独立：便于对轨迹与碰撞逻辑做单测或数值实验。

8 部署图

部署图说明程序跑在哪些机器或环境里，以及它们之间用什么方式通信。

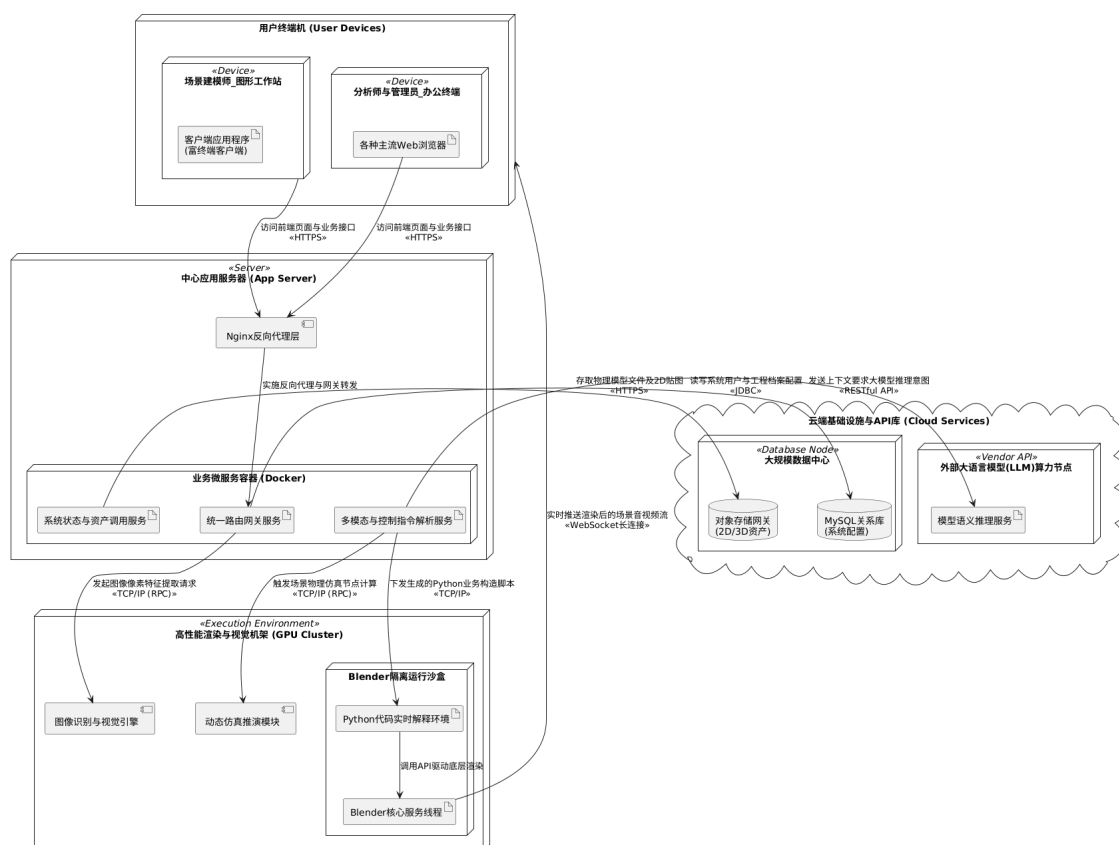


图 8.14: 部署图

8.1 部署图说明

• 用户终端

- 场景建模师使用图形工作站与富客户端：适合大场景预览与低延迟操作，常与本机或内网渲染配合。
- 行业分析师与系统管理员使用浏览器：侧重轻量访问配置、演示与后台管理。

• 中心应用服务器

- Nginx 作为反向代理，对外提供 HTTPS，向内转发到业务服务。

- Docker 容器内划分网关服务、多模态与控制解析服务、系统状态与资产调用服务等，职责分开，便于按负载扩容。
- **高性能渲染与视觉机架**
 - 图像识别与视觉引擎、动态仿真推演模块可部署在 GPU 或计算节点上，承接重计算。
 - Blender 与 Python 解释环境放在隔离沙盒中，降低插件异常对其它服务的影响。
- **云端基础设施**
 - MySQL 等关系库：存放用户、工程档案、系统配置等结构化数据。
 - 对象存储网关：存放 2D/3D 大文件。
 - 外部大语言模型节点：通过 HTTP 类接口调用，需考虑延迟、配额与失败重试。
- **主要连接关系**
 - 终端经 HTTPS 访问 Nginx 与业务层。
 - 业务服务通过 TCP/IP 或 RPC 调用视觉、仿真及 Blender 侧 Python 环境。
 - Blender 经长连接向终端推送渲染画面，满足实时预览。
 - 资产服务访问对象存储；网关服务访问关系库；解析服务访问大模型接口。
 - 上述划分便于运维分别监控 Web、渲染、存储与外部 API 的健康状态。

9 总结

在本设计报告中，我们按照用例图、活动图、类图、顺序图、协作图、状态图、构件图和部署图的顺序，从角色与功能、流程、数据结构、消息顺序、对象协作、生命周期和部署几方面说明了智能城市生成系统的设计要点，与我们课上的前一个作业”需求分析”对该系统的需求说明相一致，都可以作为后续我们进一步细化设计与实现的参考。

同时，我也学习到了如何使用 EA 软件 (Enterprise Architect) 来进行用例图、类图、顺序图等设计分析所需要的图的手动制作与绘画，让我学习掌握到了一门新技能，也锻炼了我的设计能力与实际动手能力。